Classifying Gas Transmission and Gathering Pipeline Incident Causes Using BiGRU, BiLSTM with Attention, and BERT-base

Team 1: Sakina Saifuddin, Muhammad Murtaza, Parva Shah

Project Purpose

Gas transmission and gathering pipeline incidents can occur because of corrosion, equipment failure, excavation damage, natural forces, incorrect operation, material or weld failure, and other causes. PHMSA incident reports contain narrative descriptions explaining what happened during each incident. Our goal is to build an NLP-based text classification system that reads an incident narrative and predicts its underlying cause. We will compare three neural NLP approaches to determine how effectively they classify technical pipeline incident descriptions.

Data Source

We will use the publicly available **PHMSA Gas Transmission and Gathering Incident Data** published by the U.S. Department of Transportation Pipeline and Hazardous Materials Safety Administration (PHMSA).

Dataset: PHMSA Pipeline Incident Data

The original 2010–present PHMSA dataset contains **2,053 incident reports** and **623 raw fields**. For this project, we selected six relevant fields: `REPORT_NUMBER`, `IYEAR`, `NAME`, `CAUSE`, `CAUSE_DETAILS`, and `NARRATIVE`.

Following the recommendation to augment the data, we examined the historical PHMSA Gas Transmission and Gathering files. The compatible **2002–2009** data were merged with the 2010–present records. After removing reports without usable narratives and mapping the historical cause labels to the current cause categories, the final dataset contains **3,027 labeled incident narratives across eight cause categories**. The `NARRATIVE` field is the model input and `CAUSE` is the gold label.

Preprocessing and Data Split

Narratives will be cleaned by removing unnecessary whitespace and older HTML artifacts and converting all narrative text to **lowercase**. Lowercasing is particularly important because some PHMSA narratives are written entirely in capital letters and pretrained embeddings may not represent those forms well. The dataset will use a **stratified 70% training, 15% validation, and 15% test split**, preserving the cause-category distribution across the three subsets.

Models and Team Contributions

Each team member will implement one substantive NLP model:

- **Parva:** Train a **BiGRU** classifier using pretrained **FastText** word embeddings.
- **Muhammad:** Train a **BiLSTM with attention** classifier using pretrained **FastText** embeddings.
- **Sakina:** Use **BERT-base-uncased** with a classification head. The pretrained transformer weights will remain frozen and only the classification head will be trained. BERT will use its own tokenizer and pretrained embeddings.

All members will also contribute to preprocessing, data augmentation, evaluation, model comparison, and final analysis.

Class Imbalance Experiments

Each of the three models will be evaluated using two approaches for handling the unequal number of examples among cause categories.

Experiment 1 – Class-weighted loss: Models will be trained using the original training set. Class weights will be included in the loss function so that mistakes on under-represented classes receive greater importance during optimization.

Experiment 2 – Smart oversampling: A small number of real narratives from under-represented categories will be selected from the **training data only**. An AI assistant will be asked to generate similar narratives or paraphrased versions while preserving the original cause category. These synthetic examples will be added to the training set to improve class balance. Class-weighted loss will not be used for this experiment.

The validation and test sets will remain unchanged in both experiments so that the two approaches can be compared fairly.

Evaluation and Final Deliverable

For every model and experiment, we will report **precision, recall, F1-score, and accuracy**, including per-class and **macro- and micro-averaged** results. This will allow us to compare both the three model architectures and the two class-imbalance strategies.

The best-performing model and training approach will be deployed as a **Streamlit web application**. A user will enter a new pipeline incident narrative and the application will return its predicted cause category. Development will use **Python, PyTorch, scikit-learn, VS Code, and Git/GitHub**; Google Colab may be used for GPU resources, and the final application will be hosted on **Oracle Cloud Infrastructure (OCI)**.